EKSAMENSOPPGAVE

Emnekode:	**DAT107**
Emnenavn:	**Databaser**
Dato:	**05.01.2023**

Eksamensform:	Skriftlig digital hjemmeeksamen
Eksamenstid:	09:00 – 13:00
Antall eksamensoppgaver:	7
Antall sider (inkludert denne):	9
Antall vedlegg:	0
Tillatte hjelpemidler:	Alle bortsett fra samarbeid
Emneansvarlig:	Lars-Petter Helland
Kommer til eksamenslokalet:	Nei – hjemmeeksamen

Telefonnummer emneansvarlig kan nås på under eksamen:

Lars-Petter Helland:	928 28 046
Dag Toppe Larsen:	483 07 293
Usman Ahmed	939 99 202

Merknader:

Jeg bekrefter at arbeidet er selvstendig utarbeidet, og at jeg kjenner til hva *Forskrift om studium og eksamen ved Høgskulen på Vestlandet, sier om fusk og forsøk på fusk i § 12-1.*

Oppgave 1 – Flervalg (30 poeng, ~72 min.)

Velg nøyaktig et av alternativene for hver oppgave.

1. Hva er sant om tofaselåsing
    a. Unngår vranglås
    b. To faser - først bare låsing, så bare opplåsing
    c. Gammelt begrep for vranglås
    d. Det er ingenting som heter tofaselås
2. Hva er sant om indekser
    a. Ikke lenger relevant – alle databaser bruker alltid indekser som standard på alle kolonner
    b. Brukes for å gjøre søk på kolonner raskere
    c. Spesielt viktig for fritekstsøk i kolonner
    d. Moderne datamaskiner er så raske at indekser ikke lenger er nødvendig
3. Hva er sant om XML
    a. Moderne navn på HTML
    b. Brukes hovedsakelig for å lagre binærdata som tekst
    c. Forløper for JSON, ikke lenger mye brukt
    d. Brukes for å lagre data på menneskelesbart format
4. Hva er sant om datatypen decimal
    a. Brukes hovedsakelig når man har behov for å lagre verdier med grunntall 10 med bestemt antall desimaler
    b. Gammel måte å lagre tall på, erstattet av int og float
    c. Kan bare angi totalt antall desimal som parameter
    d. Samme som int
5. Hva er sant om mange-til-mange—forhold
    a. Kan ikke skrives i kråkefot, bare i UML
    b. Må implementeres ved hjelp av koplingstabell
    c. Mange-til-mange indikeres med * på minst på en av sidene i UML
    d. Mange-til-mange bør unngås i databasedesign
6. Hva er sant om NoSQL
    a. Etterfølger til SQL – relasjonsdatabaser er ikke lenger relevante
    b. Kan i en del bruksområder være bedre egnet enn relasjonsdatabaser
    c. Det samme som XML og JSON
    d. NoSQL er egentlig relasjonsdatabaser, bare at man bruker annet spørrespråk enn SQL
7. Hva er sant om XML Schema
    a. Brukes for å angi hva som er “lov” i en XML-fil
    b. Navn på siste versjon av XML
    c. Et bibliotek for å parse XML-filer
    d. En variant av XML for å lagre geografiske data

8. Hva er sant om normalisering
    a. En database som følger alle normaliseringsreglene er garantert å tilfredsstille kravspesifikasjonene
    b. Denormalisering er det samme som NoSQL
    c. Et sett med regler som brukes for å sjekke at en database tilfredsstiller visse formelle minimumskrav
    d. Ble bare brukt i eldre relasjonsdatabaser, men ikke relevant i dag
9. Hva er sant om ER-modeller
    a. “Arkitekttegning” av database
    b. Har blitt erstattet av NoSQL
    c. Egentlig ikke nyttig for profesjonelle databasedesignere, men for å vise til oppdragsgivere
    d. Bruk av ER-modeller garanterer at database også er normalisert
10. Hva er sant om relasjonsalgebra
    a. Et spørrespråk er relasjonskomplett dersom det har samme uttrykkskraft som relasjonsalgebra
    b. Synonym for SQL
    c. Bruk av relasjonsalgebra sikrer at databasen er normalisert
    d. Relasjonsalgebra er en forløper for SQL og er ikke lenger relevant
11. Hva er sant om indre forening
    a. Tar med bare matchende rader i begge tabeller
    b. Tar med bare matchende rader i venstre tabell
    c. Tar med bare matchende rader i høyre tabell
    d. Tar med alle rader i begge tabeller
12. Hva er sant om 0..1--1..*
    a. Venstre tabell kan ha minimum 1 og maksimum mange av høyre tabell
    b. Høyre tabell kan ha minimum 1 og maksimum mange av venstre tabell
    c. Venstre tabell kan ha minimum 0 og maksimum 1 av høyre tabell
    d. Uttrykket er ugyldig
13. Hva er sant om primærnøkler
    a. Vil automatisk ha indeks i SQL
    b. Kan være NULL
    c. Kan bare være en kolonne
    d. Det samme som kandidatnøkkel
14. Hva er sant om tabeller
    a. Rekkefølge av kolonner har betydning, men ikke rekkefølge av rader
    b. Rekkefølge av rader har betydning, men ikke rekkefølge av kolonner
    c. Både rekkefølge av kolonner og rader har betydning
    d. Verken rekkefølge av kolonner eller rader har betydning

15. Hva er sant om views
    a. Annet navn for tabell
    b. Annet navn for nøstet spørring
    c. “Lagret spørring” hvis output kan brukes som input for annen spørring
    d. Tidligere brukt teknikk som ikke lenger relevant

Oppgave 2 – SQL (14 poeng, ~33,6 min.)

Ta utgangspunkt i disse tabellene:

Ansatt(<u>ansattnummer</u>, navn, kontor, telefon, epost, overordnet, avdeling*)

Avdeling(<u>avdelingskode</u>, avdelingsnavn, budsjett, avdelingsleder*)

Kolonnen Avdelingsleder inneholder ansattnummer til ansatt som er leder.

a) Skriv SQL-kode for å opprette tabellen Ansatt. Du kan anta at tabellen Avdeling allerede eksisterer. Få med datatyper, primærnøkler, fremmednøkler, on update/delete, eventuelt ikke tillatt null-merke. Du må selv vurdere hva som er hensiktsmessig. Kommenter/forklar valgene du tar.
b) Skriv en spørring som viser all informasjon om alle ansatte, samt navn på avdeling som den ansatte arbeider i.
c) Skriv en spørring som viser all informasjon om alle avdelinger, samt navn på lederen for denne avdelingen.
d) Skriv en spørring som viser all informasjon om alle ansatte, samt navn på avdeling som den ansatte arbeider i, samt navn på lederen for denne avdelingen.
e) Ta utgangspunkt i din besvarelse av delspørsmål b). Er det mulig å ha ansatte som ikke har avdeling i databasen? Grunngi svaret ditt.
f) Ta utgangspunkt i din besvarelse av delspørsmål c). Vi antar her at databasen tillater ansatte uten avdeling. Vil da besvarelsen din inkludere ansatte uten avdeling? Grunngi svaret ditt.
g) Ta utgangspunkt i din besvarelse av delspørsmål c). Skriv spørringen som relasjonsalgebra. Potensielt nyttige operatorer til å bruke i svar: π, σ, $\cup$, $\cap$, $\times$, $\bowtie$, $\ltimes$, $\rtimes$ og $\mapsto$.
h) Skriv en spørring som finner antall ansatte i hver avdeling.

Oppgave 3 – normalisering (10 poeng, ~24 min.)

En database har følgende tabell:

Kontorplass(bygning, etasje, rom, plass, bygningsansvarlig, fasttelefon, ansattnummer, navn, fødselsdato, overordnet)

Eksempeldata:

“A”, 2, 3, “b”, 43, 55667788, 4, “Kari Nordmann”, “2000-01-01", NULL

“C”, 1, 2, “a”, 23, 55668877, 4, “Kari Nordmann”, “2000-01-01", NULL

“A”, 2, 3, “b”, 43, 55667788, 10, “Ola Nordmann”, “2001-02-03", 4

a) Diskuter mulige kandidatnøkler (eventuelt sammensatt) ut fra eksisterende kolonner (ikke surrogatnøkkel/løpenummer). Kommenter mulige «problemer» med ulike kandidatnøkler, og velg til slutt primærnøkkel fra kandidatnøklene.
b) Med utgangspunkt i besvarelsen fra a), er tabellen nå på 1NF? Hvorfor/hvorfor ikke? Hvis ikke, modifiser den slik at den er på 1NF.
c) Med utgangspunkt i besvarelsen fra b), er tabellen nå på 2NF? Hvorfor/hvorfor ikke? Hvis ikke, modifiser den slik at den er på 2NF.
d) Med utgangspunkt i besvarelsen fra c), er tabellen nå på 3NF? Hvorfor/hvorfor ikke? Hvis ikke, modifiser den slik at den er på 3NF.

Oppgave 4 – modellering (10 poeng, ~24 min.)

En bedrift har behov for en database for å håndtere ansatte og prosjekt de er involvert i. Alle ansatte har ansattnummer, fornavn, etternavn, fødselsdato, ansattdato, avdeling og kontor. Videre har alle ansatte har en nærmeste leder (bortsett fra topp-sjef).

Bedriften har en rekke prosjekt. Hvert prosjekt har prosjektnummer, prosjekttittel, prosjektleder, oppstartsdato samt avsluttingsdato. Videre kan et prosjekt ha flere ansatte som arbeider på det, og hver ansatt kan arbeide på flere prosjekt. For hver ansatt som arbeider på et prosjekt skal det lagres hvor mange timer denne ansatte arbeider på det.

Tegn et ER-diagram (i en av notasjonene som har vært brukt i kurset) for denne databasen. Få med all relevant informasjon (primærnøkler, fremmednøkler, kardinalitet, sterke/svake entiteter, osv.).

Kommenter valgene du tar for å klargjøre hvordan du tenker/resonerer.

Oppgave 5 - ORM/JPA (20 poeng, ~48 min.)

Vi skal jobbe litt med en database med oversikt over biler og bileiere.

Vi tenker oss at data om biler er lagret i tabellen **bil**:

regnr (PK)	aarsmodell	type	farge
'SU 12345'	2008	'Mazda 5'	'blå'
'SV 23456'	…	…	…
'EL 34567'	…	…	…

Vi tenker oss at data om eiere er lagret i tabellen **person**:

pnr (PK)	fornavn	etternavn	mobil
'12045677777'	'Per'	'Hansen'	12345678
'23056788888'	'Knut'	'Nilsen'	23456789
…	...	...	...

Vi tenker oss at vi er interessert i historikken til eierskapene. En bil kan da ha hatt flere eiere, og en person kan eie flere biler. Data om eierskapene er lagret med **fradato** og **tildato** i tabellen **eierskap**. Tildato er NULL for "gjeldende" eierskap. Enr er en autogenerert primærnøkkel:

enr (PK)	bil_id	eier_id	fradato	tildato
1	'SU 12345'	'12045677777'	'2008-01-01'	'2012-12-31'
2	'SU 12345'	'23056788888'	'2013-01-01	NULL
3	...	...	...	...

Vi ønsker å jobbe med denne databasen i et Java-program.

a) (6% ~ 15 min) Skriv Java-klassene for entitetstypene **Bil, Person** og **Eierskap** som tilsvarer databasetabellene vist over. Du trenger ikke å skrive metoder, kun instans-/objektvariabler og JPA-annoteringer. *(Du kan i resten av spørsmålene anta at gettere, settere og toString finnes for alle tre klassene.)* Vi ønsker toveis forbindelse mellom entitetene for enklere å kunne navigere mellom dem.

Vi antar at vi har en hjelpeklasse **BileierDAO**. Du skal lage et par metoder i denne. Du kan anta at det er opprettet og at du har tilgang til en EntityManagerFactory kalt emf.

b) (2% ~ 5 min) Skriv en metode i BileierDAO som lar oss søke opp (og returnere) en bil fra databasen med et gitt registreringsnummer (gitt som parameter). Manglende treff skal returnere **null**.

c) (4% ~ 10 min) Skriv en main-metode som bruker metoden du laget i b) til å søke opp bil med registreringsnummer 'SU 12345' og skrive denne ut på skjermen som vist i eksemplet under.

```
Bil SU 12345, en blå Mazda 5, år 2008
Eierskap:
   2008-01-01 .. 2012-12-31      - Per Hansen (mob 12345678)
   2013-01-01 .. dd              - Knut Nilsen (mob 23456789)
```

Du kan anta at Bil sin toString() returnerer en slik:

```
Bil SU 12345, en blå Mazda 5, år 2008
```

og at Eierskap sin toString() returnerer en slik:

```
2008-01-01 .. 2012-12-31  - Per Hansen (mob 12345678)
```

d) (8% ~ 20 min) Skriv en metode i BileierDAO som registrerer eierskifte. Parametre til metoden skal være **regnr** for aktuell bil, **pnr** til ny eier, og **dato** for eierskiftet. For enkelhets skyld antar vi at bilen finnes, personen finnes og at det finnes ett nåværende eierskap for bilen der tildato er NULL i databasen. Metoden skal så sette tildato på gammelt eierskap, og opprette et nytt eierskap fra oppgitt dato.

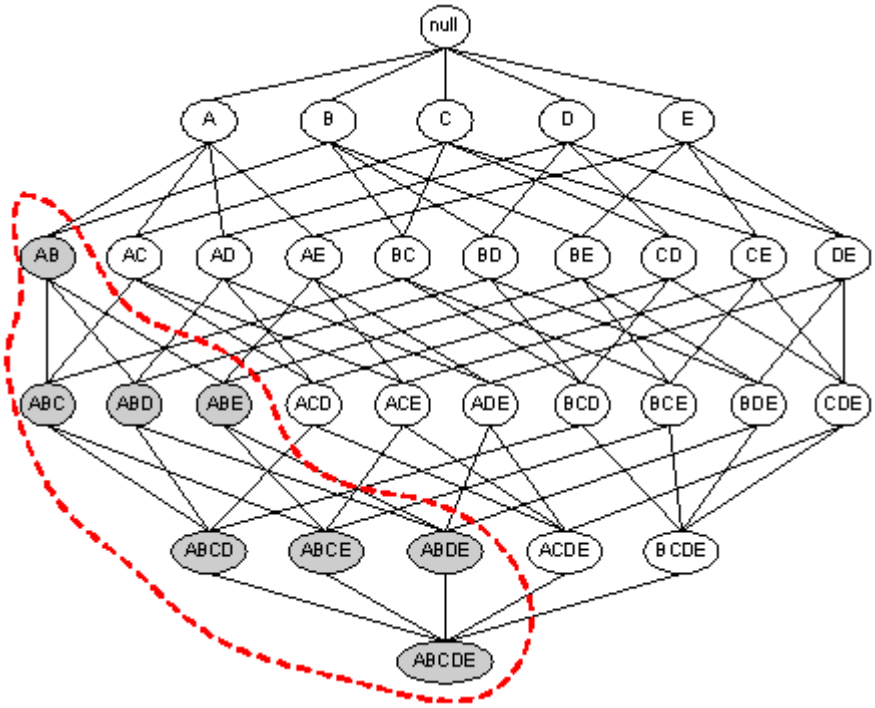
Oppgave 6 - datamining (10 poeng, ~24 min.)

a) Hva er resultatene av å anvende Apriori-algoritmen på datasettet i Tabell1 med følgende valg?
   1) Bruk 0,2 for “minimum support” verdi for å finne to settet av “frequent k-itemsets" (k>=1)
   2) Bruk 0,7 for “confidence” for å finne settet av “association rules”.

Trans ID	**Ting kjøpt**
101	milk, bread, eggs
102	milk, juice
103	juice, butter
104	milk, bread, eggs
105	coffee, eggs
106	coffee
107	coffee, juice
108	milk, bread, cookies, eggs
109	cookies, butter
110	milk, bread

Tabell 1 Transaksjonstabell

b) Ta utgangspunkt i Figur 1 og besvar følgende spørsmål.
   1) Hvordan vil “combinatorial explosions” påvirke “computational efficiency” og tiden påkrevd for “high utility itemset mining”?
   2) Hvilke teknikker har blitt utviklet for å håndtere disse ”issues”?



Figur 1 “Issues” for “high utility itemset mining”

Oppgave 7 – NoSQL (6 poeng, ~14,4 min.)

a) Ta utgangspunkt i oppgave 2 (SQL). Skriv en "rad" fra hver av tabellene som XML.
b) Ta utgangspunkt i oppgave 2 (SQL). Skriv en "rad" fra hver av tabellene som JSON
c) Både XML og JSON kan brukes til å serialisere databaser i tekstform. Sammenlign formatene, og angi det du mener er eventuelle styrker og svakheter ved hver av dem. Dersom du skulle serialisere i database i tekstform, hvilken av formatene ville du valgt? Grunngi svarene dine.